



Universidade do Minho
Escola de Engenharia

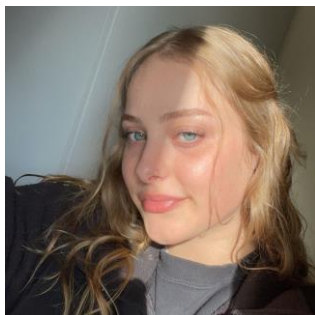
LICENCIATURA EM ENGENHARIA E GESTÃO DE SISTEMAS DE INFORMAÇÃO

Semestre 2 2022/2023

Técnicas de Inteligência Artificial
-TIA-

PROJETO 1

Grupo 6



97042 - Ana Luísa Silva

a97042@alunos.uminho



95601 - André Macedo Barros

a95601@alunos.uminho.pt



91663 - Carlos Rodrigues

a91663@alunos.uminho.pt

Índice

1. Introdução.....	1
1.1 Enquadramento	1
1.2 Objetivos	1
2. Execução do Projeto	2
Projeto 1 (P1) – Recomendação de Meio de Transporte.....	4
3. Parte A - Aquisição manual de conhecimento	4
3.1 Interface.....	4
3.2. Base de Conhecimento	5
3.3 Base de Dados	6
4. Parte B: Aquisição automática de conhecimento	7
5. Conclusões	9
5.1 Síntese	9
5.2 Discussão.....	9
5.3 Funcionamento do Trabalho em Grupo	9
Anexos.....	10
Anexo A.....	10
Código Parte A	10
Código Parte B	16
Anexo B.....	23
Contrato do Grupo	23
Bibliografia	24

1. Introdução

1.1 Enquadramento

No contexto da unidade curricular de Tecnologias de Inteligência Artificial, do terceiro ano do curso Licenciatura em Engenharia e Gestão de Sistemas de Informação, foram-nos propostos a realização de dois projetos.

Este primeiro projeto, tem como objetivo desenvolver um sistema baseado em conhecimento (SBC), que é um sistema de inteligência artificial que utiliza conhecimentos prévios para realizar tarefas específicas ou tomar decisões. Este armazena e manipula informações estruturadas, adquiridas através de várias fontes, como perguntas ao utilizador ou de outras, e utiliza regras de inferência para resolver problemas complexos em áreas como medicina, engenharia, finanças, entre outras. Esses sistemas são projetados para automatizar tarefas, fornecer suporte à tomada de decisões e oferecer respostas e informações precisas com base em um conjunto de conhecimentos especializados.

O SBC em causa, terá como foco a recomendação de meios de transporte em linguagem Prolog. Esse sistema será um programa capaz de manipular conhecimento e informações de forma inteligente, fornecendo orientações ao utilizador sobre as melhores soluções segundo determinados parâmetros. Para este trabalho, escolhemos utilizar a ferramenta SWI-Prolog para desktop, cujo funcionamento foi ensinado nas aulas lecionadas ao longo do semestre.

1.2 Objetivos

Conforme mencionado anteriormente, para a parte A o objetivo proposto foi o desenvolvimento um Sistema Baseado em Conhecimento (SBC) que operará através de um processo iterativo, por meio da formulação de perguntas ao utilizador, derivando, assim, conclusões que possibilitarão a construção da recomendação do meio transporte mais adequado consoante os parâmetros estabelecidos no desenvolvimento dos programas.

O programa será dividido em 4 ficheiros, sendo estes a base de dados, sistema de inferência, base de conhecimento e interface.

Relativamente à parte B, o objetivo será a geração de uma nova base de conhecimento através da aquisição automática de conhecimento, este será feito por via do software "RapidMiner".

2. Execução do Projeto

Planeamento do Projeto

☐ Início	6 days? 4/3/23 8:00 AM	4/10/23 5:00 PM			
Estudo do Funcionamento de um SBC	6 days? 4/3/23 8:00 AM	4/10/23 5:00 PM		Ana Silva;André Barros;Carlos Rodrigues	
Estudo do Funcionamento do Prolog	6 days? 4/3/23 8:00 AM	4/10/23 5:00 PM		Ana Silva;André Barros;Carlos Rodrigues	
☐ Projeto 1 - Parte A	10 days? 4/11/23 8:00 AM	4/24/23 5:00 PM	1		
Contribuição Base de Dados	1 day? 4/11/23 8:00 AM	4/11/23 5:00 PM		André Barros	
Construção Interface	1 day? 4/11/23 8:00 AM	4/11/23 5:00 PM		Carlos Rodrigues	
Adaptação Sistema Inferência	1 day? 4/11/23 8:00 AM	4/11/23 5:00 PM		Ana Silva	
Construção Base de Conhecimento Manual	6 days? 4/15/23 8:00 AM	4/24/23 5:00 PM		Ana Silva;André Barros;Carlos Rodrigues	
☐ Projeto 1 - Parte B	8 days? 4/25/23 8:00 AM	5/4/23 5:00 PM	4		
Estudo do funcionamento do Rapid Miner	4 days? 4/25/23 8:00 AM	4/28/23 5:00 PM		Ana Silva;André Barros;Carlos Rodrigues	
Construção Base de conhecimento Automatica	4 days? 5/1/23 8:00 AM	5/4/23 5:00 PM	10	Ana Silva;André Barros;Carlos Rodrigues	
☐ Fim	1 day? 5/9/23 8:00 AM	5/9/23 5:00 PM	9		
Elaboração do Relatório	1 day? 5/9/23 8:00 AM	5/9/23 5:00 PM		Ana Silva;André Barros;Carlos Rodrigues	

Durante a realização deste projeto, encontramos-nos perante um desafio que se distinguia significativamente dos trabalhos aos quais estávamos habituados. Fomos confrontados com um novo paradigma de programação lógica, o que nos obrigou a adquirir um novo conjunto de competências e conhecimentos. Compreender e aplicar esse paradigma requiriu um esforço adicional da nossa parte, mas estávamos determinados a superar essa barreira.

Uma das principais dificuldades que enfrentamos foi a constante necessidade de refazer várias partes do projeto. Isso gerou um ambiente de incerteza e tornou a divisão de tarefas um desafio contínuo. Foi preciso encontrar um equilíbrio entre a definição de tarefas específicas e a flexibilidade necessária para nos adaptarmos às mudanças constantes. Por vezes, tivemos que abandonar o conceito tradicional de atribuição de responsabilidades individuais e optar por uma abordagem colaborativa, em que todos trabalhavam em conjunto para alcançar os melhores resultados possíveis.

Essa colaboração intensa foi essencial para maximizar o aproveitamento do resultado. Estabelecemos uma comunicação aberta e transparente, partilhando ideias e conhecimentos, debatendo soluções e apoiando-nos mutuamente ao longo do processo. Essa abordagem permitiu-nos aproveitar ao máximo as competências individuais de cada membro da equipa, resultando numa sinergia que impulsionou o progresso e a qualidade do trabalho desenvolvido.

Apesar dos desafios e da necessidade de adaptação constante, o nosso empenho coletivo e a dedicação demonstrada possibilitaram a obtenção de um resultado que estamos orgulhosos. O trabalho diferenciado, a compreensão do novo paradigma e a colaboração estreita foram fatores determinantes para superarmos os obstáculos encontrados e maximizar o aproveitamento do resultado.

Posto isto, a contribuição de cada elemento foi a seguinte:

Ana Luísa Silva:

Parte A:

- Análise e pesquisa sobre o problema.
- Criação da base de conhecimento.
- Elaboração da interface para o utilizador.

Parte B:

- Criação do documento Excel.
- Criação da base de conhecimento via aquisição automática de conhecimento.

Relatório:

- Estruturação do relatório.
- Explicação do funcionamento da parte B.

André Barros:

Parte A:

- Análise e pesquisa sobre o problema.
- Elaboração da base de conhecimento.

Parte B:

- Criação do documento Excel.
- Criação da base de conhecimento via aquisição automática de conhecimento.

Relatório:

- Enquadramento.
- Explicação do funcionamento da parte A

Carlos Rodrigues:

Parte A:

- Análise e pesquisa sobre o problema.
- Elaboração da base de dados.
- Aplicação de Sistema de Inferência.

Parte B:

- Criação do documento Excel.
- Criação da base de conhecimento via aquisição automática de conhecimento.

Relatório:

- Elaboração do contrato de grupo.
- Elaboração do diagrama de Gantt.

Projeto 1 (P1) – Recomendação de Meio de Transporte

3. Parte A - Aquisição manual de conhecimento

Na primeira parte do Projeto 1, produziu-se um SBC que aconselha e orienta o utilizador através de um processo interativo. Após responder algumas questões, será sugerido o meio de transporte mais adequado às características da viagem pretendida.

Para tal, foi criada uma base de conhecimento, que, através de um conjunto de regras, relaciona factos “intermédios” (respostas dadas pelo utilizador), a factos “finais”, sendo estes os factos que estão definidos na Base de Dados, de modo a ligar o “perfil” de características do viajante, com a melhor solução decidida pelo programa, posto isto, foi definido um conjunto seletivo de perguntas em conformidade com a base de conhecimento e base de dados.

3.1 Interface

Na interface, foram definidas as perguntas que vão ser feitas ao utilizador, de modo a obter os factos necessários para a chegar à conclusão, as perguntas que o utilizador terá de responder são:

- **Necessita de ajuda para recomendação de meio de transporte?**
- **Qual o seu local de partida?**
- **Qual o seu destino?**
- **Qual o seu objetivo de viagem?**

Neste ponto, a próxima questão será diferente segundo a resposta do utilizador, sendo as opções: Lazer ou Trabalho.

<LAZER>

- **Quanto tempo vai ficar hospedada?**

<TRABALHO>

- **Pretende usar computador durante a viagem?**

Posto isto, a sequência de perguntas volta a seguir um caminho único:

- **E urgente?**
- **Quanto está disposto a gastar?**
- **Fuma?**
- **Quantas pessoas vão consigo na viagem?**

3.2. Base de Conhecimento

Na base de conhecimento, foi criado um conjunto de regras de inferência para o sistema de recomendação de meio de transporte. Essas regras estabelecem relações lógicas entre as preferências do utilizador e as características do meio de transporte recomendado. As relações criadas foram as seguintes:

- Se a viagem for de trabalho e for urgente, então recomende um meio de transporte rápido.
- Se a viagem for de trabalho e não for urgente, então recomende um meio de transporte mais lento.
- Se a viagem for de trabalho e o preço for baixo, então a recomendação final é um meio de transporte barato.
- Se a viagem for de trabalho e o utilizador precisar de utilizar um computador, então recomende um meio de transporte com acesso Wi-Fi.
- Se a recomendação for um meio de transporte lento e o utilizador precisar de acesso Wi-Fi, então a recomendação final é um meio de transporte com tomada elétrica.
- Se a recomendação for um meio de transporte rápido e o utilizador precisar de acesso Wi-Fi, então a recomendação final é um meio de transporte com conectividade online.
- Se o utilizador sofrer de enjoos, então a recomendação final é um meio de transporte confortável.
- Se a viagem for de trabalho e houver mais de 5 pessoas, então a recomendação final é um meio de transporte com mais de 5 lugares disponíveis.
- Se a viagem for de trabalho e houver no máximo 5 pessoas, então a recomendação final é um meio de transporte com no máximo 5 lugares disponíveis.
- Se a viagem for de trabalho e houver mais de 5 lugares disponíveis, então a recomendação final é utilizar transporte público.
- Se o utilizador for fumador e houver no máximo 5 lugares disponíveis, então a recomendação final é utilizar transporte pessoal.
- Se a viagem for de lazer e for urgente, então recomende um meio de transporte rápido.
- Se a viagem for de lazer e não for urgente, então recomende um meio de transporte mais lento.
- Se a viagem for de lazer e o preço for baixo, então a recomendação final é um meio de transporte barato.
- Se a viagem for de lazer e houver mais de 5 pessoas, então a recomendação final é um meio de transporte com mais de 5 lugares disponíveis.
- Se a viagem for de lazer e houver no máximo 5 pessoas, então a recomendação final é um meio de transporte com no máximo 5 lugares disponíveis.
- Se a viagem for de lazer e houver mais de 5 lugares disponíveis, então a recomendação final é utilizar transporte público.
- Se a viagem for de lazer e houver necessidade de bagagem de porão, então a recomendação final é escolher um meio de transporte que permita bagagem de porão.

- Se a recomendação for um meio de transporte rápido e houver necessidade de bagagem de mão, então a recomendação final é escolher um meio de transporte que permita bagagem de mão.

Estas regras adicionais complementam o sistema de recomendação existente, fornecendo mais critérios para a seleção do meio de transporte adequado com base nas preferências e necessidades do utilizador.

3.3 Base de Dados

Finalmente o sistema compara os factos finais, que são guardados numa lista e depois analisados com base nas informações pré-definidas de origem, destino, custo e tempo, associados a cada um dos transportes.

Por exemplo, a entrada:

`bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,porto,30,40).`

Esta indica que se as características deduzidas forem: “barato, confortável, transporte_publico, bagagemDePorao”, para uma viagem de Braga ao Porto, o transporte mais indicado será o comboio, com um tempo de duração de 30 minutos e um custo de 40€.

Requisitos	Descrição	Efetinado
Requisitos Mínimos	A solução (gerada pelo SBC) terá de ser implementada na linguagem Prolog via regras de produção geradas via aquisição manual de conhecimento.	X
	O SBC deverá aconselhar e orientar o utilizador através de um processo interativo que lhe permitirá, após responder a algumas questões, chegar a um (ou vários) cenário(s) possível para aconselhamento	X
	Seguir a metodologia de desenvolvimento de SBC estudada (4 fases) e as técnicas de aquisição de conhecimento manuais e automáticas que achar mais convenientes	X
	Separar (em ficheiros distintos ou identificar claramente por comentários): base de conhecimento, sistema de inferência, base de dados e interface.	X
Requisitos Opcionais	Sistema de inferência considerando incerteza	
	Sistema de inferência considerando explicação	
	Interface amigável para o utilizador	X

4. Parte B: Aquisição automática de conhecimento

A realização desta parte do projeto é bastante semelhante à primeira parte do projeto, sendo que, dessa forma, conseguimos aproveitar grande parte do trabalho que já tinha sido realizado. Contudo, tivemos de fazer algumas alterações no que concerne à base de conhecimento.

Para a obtenção da base de conhecimento, contrariamente ao realizado no passo anterior que foi realizado de forma manual, nesta fase utilizamos o RapidMiner Studio, que nos permitiu obter um conjunto de regras correspondentes à base de conhecimento, após termos colocado os dados num ficheiro Excel. A combinação utilizada no RapidMiner foi a seguinte:

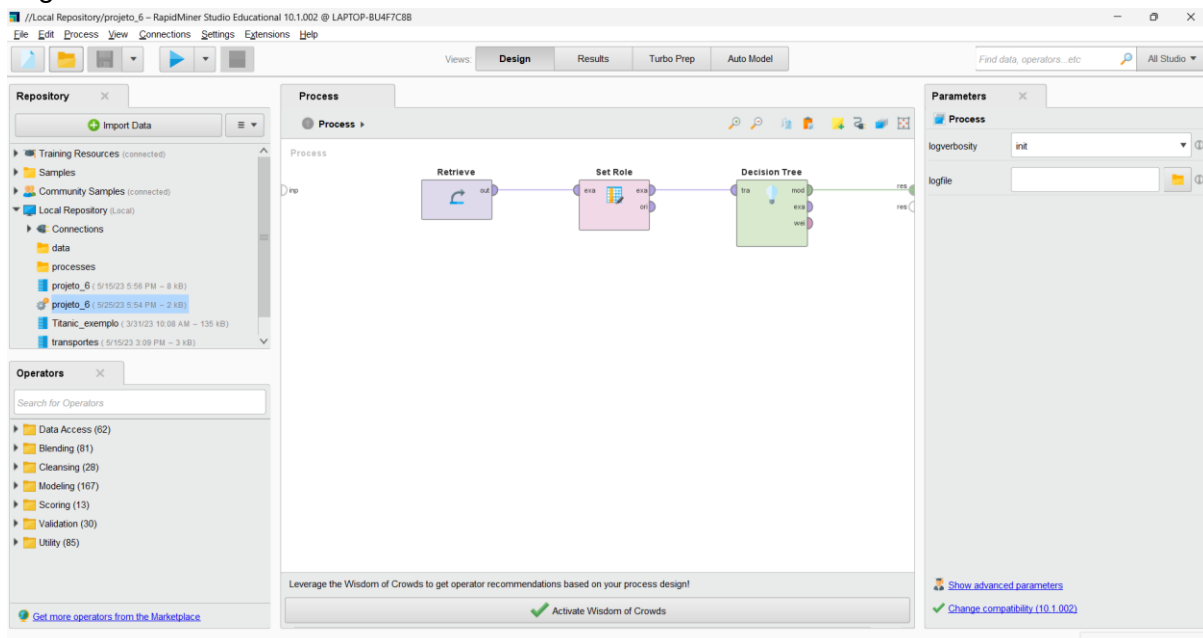


Figura 1 - Processo RapidMiner

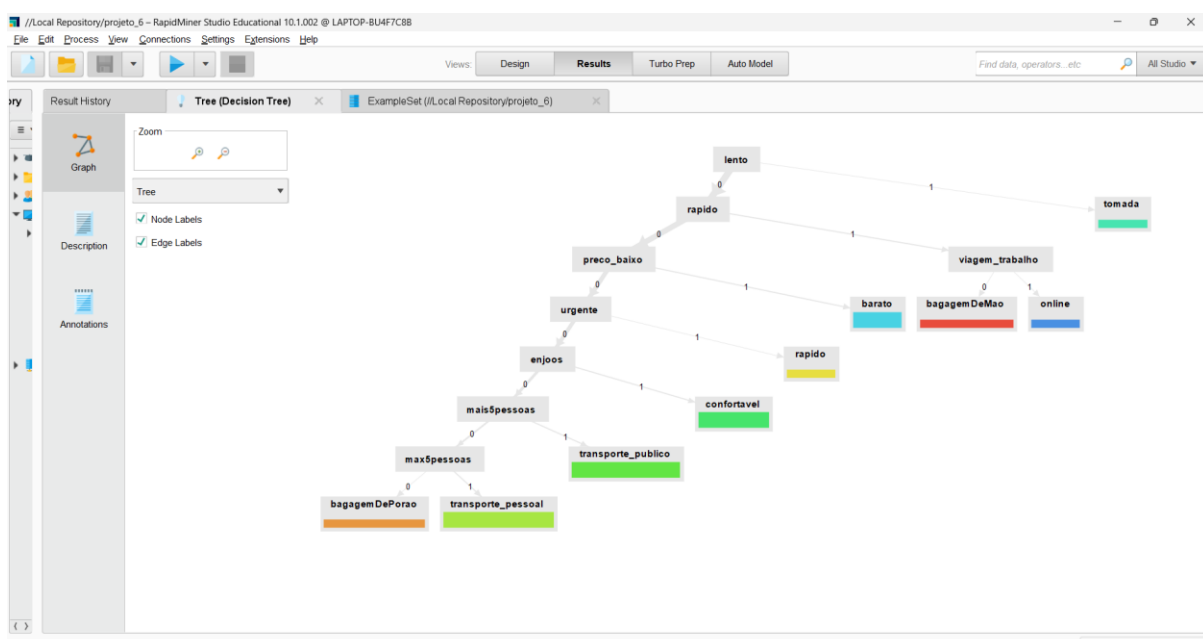


Figura 2 - Árvore RapidMiner

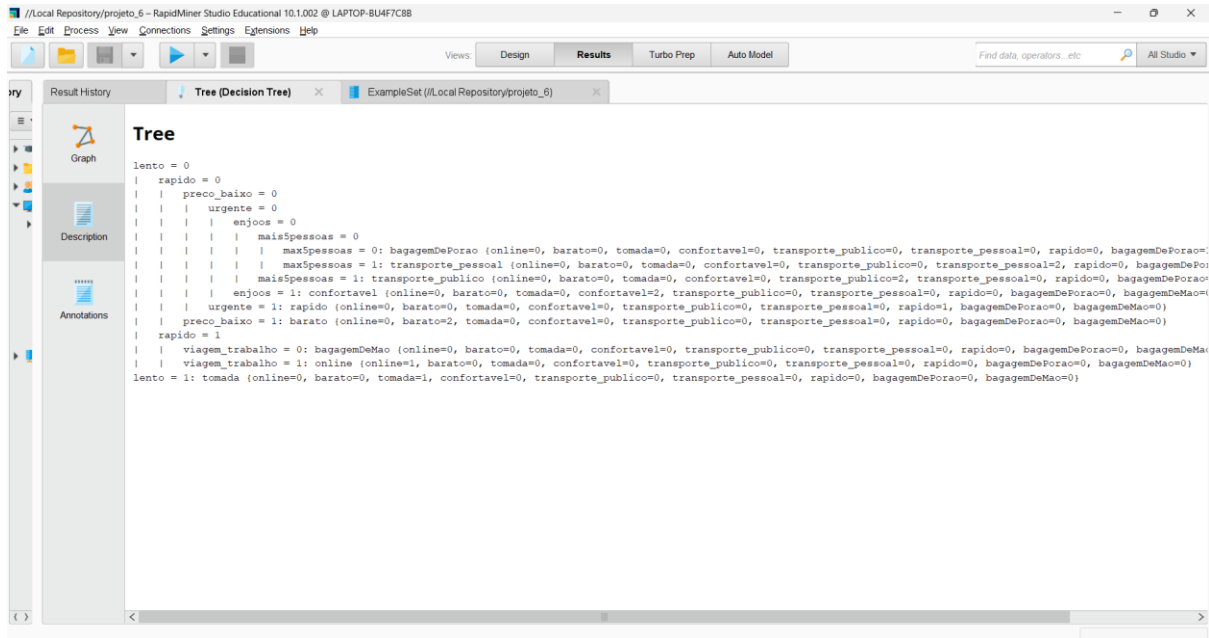


Figura 3 - Regras de Inferência

Requisitos	Descrição	Efetuação
Requisitos	A solução terá de ser implementada na linguagem Prolog via regras de produção geradas via uma aquisição de conhecimento automática (Data Mining).	X
	Os métodos de extração automática podem ser obtidos via Prolog e/ou via ferramentas de Data Mining (e.g. R, Python, Rapidminer, Weka).	X
	Podem ainda processar/preparar os dados com quaisquer em Prolog via regras de produção, por exemplo via esforço de codificação manual.	X

5. Conclusões

5.1 Síntese

O programa elaborado cumpre todos os requisitos delineados no enunciado do Projeto 1 (V3), sendo assim o nosso programa capaz de recomendar o meio de transporte mais adequado para um utilizador, tirando partido das capacidades de um SBC para inferir conclusões através de factos estabelecidos por perguntas realizadas ao utilizador. Na Parte A o programa utiliza uma base de conhecimento que foi adquirida através da metodologia de 4 fases estudada, sendo na Parte A através de aquisição manual, e na Parte B aquisição automática com adaptação a regras Prolog utilizando esforço de codificação manual.

Ambas partes funcionam utilizando um sistema de Forward Chaining, que utiliza factos conhecidos para derivar novos factos e uma conclusão final, que é apresentada ao utilizador.

Por fim, a interface com o utilizador é feita através da consola Prolog (XPCE), escolhida por ser uma forma conveniente de codificar e razoavelmente intuitiva para um utilizador final navegar.

Com a realização deste projeto, foi possível compreender o funcionamento de um SBC, tendo sido aplicados com sucesso todos os métodos e conhecimentos adquiridos nas aulas para construir um sistema robusto, e arbitrariamente mais importante foi possível descobrir um novo paradigma de programação, relativamente diferente dos “tradicionais

5.2 Discussão

Tendo em conta que todos os objetivos propostos foram atingidos, de forma completa e fiável, consideramos o programa desenvolvido como perfeitamente adequado para a tarefa que é pretendida desempenhar. Apesar disto, sentimos dificuldades em adaptarmo-nos a este novo paradigma de programação, que como referido anteriormente é vastamente diferente de outros paradigmas tradicionais, especialmente tendo em conta o que foi lecionado ao longo da Licenciatura. Estas dificuldades foram, no entanto, superadas tentando ignorar o intuito de desenvolver um programa imperativamente, e focando no conteúdo que foi lecionado ao longo do semestre.

5.3 Funcionamento do Trabalho em Grupo

O grupo funcionou de forma muito satisfatória, sendo que todos os elementos cumpriram as suas tarefas e seguiram o contrato de trabalho. Todos os elementos revelaram-se dispostos a trabalhar e a ajudar, o que criou um ambiente fácil de coexistir e de conseguir criar um trabalho positivo.

Assim, visto que não houve desequilíbrios de contribuição e consideramos a qualidade da solução muito boa, sugerimos as seguintes avaliações:

Ana:16

André:16

Carlos:16

Grupo: 16

Anexos

Anexo A

Código Parte A

SISTEMA DE INFERÊNCIA FORWARD CHAINING - "forward.pl"

```
%a simple backward chaining rule interpreter
:-op(800, fx, if).
:-op(700, xfx, then).
:-op(300, xfy, or).
:-op(500, xfy, and).
:-op( 800, xfx, <=).

% Simple forward chaining in Prolog
demo:-new_derived_fact(P), !, assert(fact(P)),demo.
demo:-write('Obrigado pela sua preferencia!').
new_derived_fact(Concl):-if Cond then Concl, \+ fact(Concl), composed_fact(Cond).

composed_fact(Cond):- fact(Cond).
composed_fact(Cond1 and Cond2):-
    composed_fact(Cond1),
    composed_fact(Cond2).
composed_fact(Cond1 or Cond2):-
    composed_fact(Cond1);
    composed_fact(Cond2).
```

BASE DE CONHECIMENTO - "bc.pl"

```
if viagem_trabalho and urgente then rapido.
if viagem_trabalho and n_urgente then lento.
if viagem_trabalho and preco_baixo then final:barato.
if viagem_trabalho and computador then wifi.
if lento and wifi then final:tomada.
if rapido and wifi then final:online.
if enjoos then final:confortavel.
if viagem_trabalho and mais5pessoas then mais5lugares.
if viagem_trabalho and max5pessoas then max5lugares.
if viagem_trabalho and mais5lugares then final:transporte_publico.
if fumador and max5lugares then final:transporte_pessoal.

if viagem_lazer and urgente then rapido.
```

```

if viagem_lazer and n_urgente then lento.
%if enjoos then final:confortavel.
if viagem_lazer and preco_baixo then final:barato.
if viagem_lazer and mais5pessoas then mais5lugares.
if viagem_lazer and max5pessoas then max5lugares.
if viagem_lazer and mais5lugares then final:transporte_publico.
%if fumador and max5lugares then final:transporte_pessoal.
if viagem_lazer and estadia then final:bagagemDePorao.
if rapido and estadia then final:bagagemDeMao.

```

BASE DE DADOS- "bd.pl"

```

bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,porto,30,40).
bd([barato,tomada,transporte_publico,bagagemDePorao],autocarro,braga,porto,20,30).
bd([barato,online,confortavel,transporte_pessoal,bagagemDeMao],carro,braga,porto,20,25).
bd([confortavel,transporte_publico,bagagemDeMao],aviao,braga,porto,20,70).
bd([online,confortavel,transporte_privado],taxi,braga,porto,20,50).

```

```

bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,faro,110,60).
bd([barato,tomada,transporte_publico,bagagemDePorao],autocarro,braga,faro,130,80).
bd([barato,online,confortavel,transporte_pessoal,bagagemDeMao],carro,braga,faro,100,80).
bd([confortavel,transporte_publico,bagagemDeMao],aviao,braga,faro,90,150).
bd([online,confortavel,transporte_privado],taxi,braga,faro,100,130).

```

```

bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,guimaraes,20,30).
bd([barato,transporte_publico,bagagemDePorao],autocarro,braga,guimaraes,20,25).
bd([barato,online,confortavel,transporte_pessoal,bagagemDeMao],carro,braga,guimaraes,15,20).
bd([confortavel,transporte_publico,bagagemDeMao],aviao,braga,guimaraes,10,60).
bd([online,confortavel,transporte_privado],taxi,braga,guimaraes,15,40).

```

```

bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,aveiro,70,40).
bd([barato,transporte_publico,bagagemDePorao],autocarro,braga,aveiro,70,35).
bd([barato,online,confortavel,transporte_pessoal,bagagemDeMao],carro,braga,aveiro,65,60).
.
bd([confortavel,transporte_publico,bagagemDeMao],aviao,braga,aveiro,50,90).
bd([online,confortavel,transporte_privado],taxi,braga,aveiro,65,70).

```

```

bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,lisboa,80,45).
bd([barato,tomada,transporte_publico,bagagemDePorao],autocarro,braga,lisboa,75,50).
bd([barato,online,confortavel,transporte_pessoal,bagagemDeMao],carro,braga,lisboa,70,60).
.
bd([confortavel,transporte_publico,bagagemDeMao],aviao,braga,lisboa,50,95).
bd([online,confortavel,transporte_privado],taxi,braga,lisboa,70,90).

```

```

bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,coimbra,55,40).
bd([barato,tomada,transporte_publico,bagagemDePorao],autocarro,braga,coimbra,55,42).
bd([barato,online,confortavel,transporte_pessoal,bagagemDeMao],carro,braga,coimbra,50,66).
bd([confortavel,transporte_publico,bagagemDeMao],aviao,braga,coimbra,44,99).
bd([online,confortavel,transporte_privado],taxi,braga,coimbra,50,88).

```

INTERFACE - “interface.pl”

```
:-dynamic(fact/1).
```

```
:-[bd,bc,proof,forward].
```

```
%:-initialization(interface).
```

```
interface:- retractall(fact(_)), nl,nl,write('Bem-vindo ao nosso sistema baseado em
conhecimento!!'),nl,
```

```
write('Necessita de ajuda para recomendacao de meio de transporte?'),nl,
```

```
write(' "1." - Sim, ajude-me'),nl,
```

```
write('"0." - Nao. Quero sair '),nl,read(A),(
```

```
(1 == A), nl,origem,nl;
```

```
(0 == A), nl,write('Terminou'),halt).
```

```
%braga, porto,aveiro,guima,viseu,lisboa,faro,coimbra,
```

```
origem:- nl,nl,write('Qual o seu local de partida?'),nl,
```

```
write(' "1." - Braga '),nl,
```

```
write(' "2." - Porto '),nl,
```

```
write(' "3." - Faro '),nl,
```

```
write(' "4." - Guimaraes '),nl,
```

```
write(' "5." - Aveiro '),nl,
```

```
write(' "6." - Lisboa '),nl,
```

```
write(' "7." - Coimbra '),nl,
```

```
read(B),(
```

```
(1 == B), nl,assert(origem(braga)),destino,nl;
```

```
(2 == B), nl,assert(origem(porto)),destino,nl;
```

```
(3 == B), nl,assert(origem(faro)),destino,nl;
```

```
(4 == B), nl,assert(origem(guimaraes)),destino,nl;
```

```
(5 == B), nl,assert(origem(aveiro)),destino,nl;
```

```
(6 == B), nl,assert(origem(lisboa)),destino,nl;
```

```
(7 == B), nl,assert(origem(coimbra)),destino,nl;
```

```
(otherwise), nl,write('Introduza uma opcao valida, por favor comece de novo!'),nl,nl,
```

```
origem).
```

```
destino:- nl,nl,write('Qual o seu destino? '),nl,
write(' "1." - Braga '),nl,
write(' "2." - Porto '),nl,
write(' "3." - Faro '),nl,
write(' "4." - Guimaraes '),nl,
write(' "5." - Aveiro '),nl,
write(' "6." - Lisboa '),nl,
write(' "7." - Coimbra '),nl,
```

```
read(C),(
    (1 == C), nl,assert(destino(braga)),tipo_viagem,nl;
    (2 == C), nl,assert(destino(porto)),tipo_viagem,nl;
    (3 == C), nl,assert(destino(faro)),tipo_viagem,nl;
    (4 == C), nl,assert(destino(guimaraes)),tipo_viagem,nl;
    (5 == C), nl,assert(destino(aveiro)),tipo_viagem,nl;
    (6 == C), nl,assert(destino(lisboa)),tipo_viagem,nl;
    (7 == C), nl,assert(destino(coimbra)),tipo_viagem,nl;
    (otherwise), nl,write('Introduza uma opcao valida, por favor tente
novamente!'),nl,nl, destino).
```

```
tipo_viagem:-nl,nl,write('Qual o seu objetivo de viagem?'),nl,
write(' "1." - Trabalho'),nl,
write(' "2." - Lazer'),nl,
```

```
read(D),(
    (1 == D), assert(fact(viagem_trabalho)),pc,nl;
    (2 == D), assert(fact(viagem_lazer)),estadia,nl;
    (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,
estadoViajante).
```

```
estadia:-nl,nl,write('Quanto tempo vai ficar hospedada?'),nl,
write('1. - Mais de uma semana'),nl,
write('2. - menos de uma semana '),nl,
```

```
read(G),(
    (1 == G), assert(fact(bagagem)),estadoViajante;
    (2 == G), assert(fact(n_bagagem)),estadoViajante;
    (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,
estadia).
```

```
pc:-nl,nl,write('Pretende usar computador durante a viagem?'),nl,
write('1. - Sim'),nl,
write('2. - Nao '),nl,
```

```
read(G),(
```

```
(1 == G), assert(fact(computador)),estadoViajante;  
(2 == G), assert(fact(n_computador)),estadoViajante;  
(otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl, pc).
```

```
estadoViajante:-nl,nl,write('E urgente?'),nl,  
  write(' 1.' - Quero chegar ao destino o mais cedo possivel'),nl,  
  write(' 2.' - Nao, tenho tempo'),nl,
```

```
read(D),(  
  (1 == D), assert(fact(urgente)),preco,nl;  
  (2 == D), assert(fact(n_urgente)),preco,nl;  
  (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,  
  estadoViajante).
```

```
preco:-nl,nl,write('Quanto e que esta disposto a gastar?'),nl,  
  write('1. - O minimo possivel '),nl,  
  write('2. - Isso nao e problema para mim'),nl,
```

```
read(E),(  
  (1 == E), assert(fact(preco_baixo)),enjoo;  
  (2 == E), assert(fact(preco_alto)),enjoo;  
  (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,  
  preco).
```

```
enjoo:-nl,nl,write('Tem enjoos?'),nl,  
  write('1. - Sim'),nl,  
  write('2. - Nao'),nl,
```

```
read(F),(  
  (1 == F), assert(fact(enjoos)),fumador;  
  (2 == F), assert(fact(n_enjoos)),fumador;  
  (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,  
  enjoo).
```

```
fumador:-nl,nl,write('Fuma?'),nl,  
  write('1. - Sim sou fumador'),nl,  
  write('2. - Nao'),nl,
```

```
read(H),(  
  (1 == H), assert(fact(fumador)),numero_pessoas;  
  (2 == H), assert(fact(n_fumador)),numero_pessoas;  
  (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,  
  fumador).
```

```
numero_pessoas:-
```



```

nl, write('Quantas pessoas vao consigo na viagem? '), nl,
read(N),
(N > 5, assert(fact(mais5pessoas)), resultado;
N =< 5, assert(fact(max5pessoas)), resultado;
(otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,
numero_pessoas).

```

resultado :- nl,nl,demo, comparar.

```

comparar :-
    origem(Origem),
    destino(Destino),
    findall(bd(L, M, Origem, Destino, Distancia, Custo), bd(L, M, Origem,
Destino, Distancia, Custo), K),
    resposta(K), fim.

```

```

resposta([]) :-
    write('Esperemos que goste da recomendacao, aproveite!'), nl, write('Se nao houver
recomendacoes disponiveis no momento, tente novamente mais tarde.').

```

```

resposta([bd(X, T, O, D, M, C) | R]) :-
    findall(B, fact(final:B), L),
    subset(L, X), nl, nl,
    write('_____'), write('Meio de transporte mais recomendado '), write('_____
'), nl, nl, write('_____'), write(T), nl, nl, nl,
    write('-> '), write('Origem: '), write(O), nl, nl,
    write('-> '), write('Destino: '), write(D), nl, nl,
    write('-> '), write('Distancia: '), write(M), nl, nl,
    write('-> '), write('Custo: '), write(C), nl, nl, nl,

```

resposta(R).

```

resposta([_|R]) :-
    resposta(R).

```

```

fim :-
    retractall(fact(_)),
    retractall(origem(_)),
    retractall(destino(_)), nl, nl, nl,
    write('Obrigado pela sua preferencia! Deseja ser recomendado novamente?'), nl, nl,
    write(' "1." - Sim, por favor'), nl,
    write(' "0." - Nao. Quero sair '), nl,

```

```

read(Z),(
    (1 == Z), nl, retractall(fact(_)), origem;
    (0 == Z), nl, write('Terminou'), halt;
    (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,
fim).

```

PROOF - “proof.pl”

```

% demo( P, Proof), where Proof is a proof that P is true
:- op( 800, fx, if).
:- op( 700, xfx, then).
:- op( 300, xfy, or).
:- op( 500, xfy, and).
:- op( 800, xfx, <=).

```

```

demo( P, P) :- fact( P).
demo( P, P <= CondProof) :-
    if Cond then P, demo( Cond, CondProof).
demo( P1 and P2, Proof1 and Proof2) :-
    demo( P1, Proof1), demo( P2, Proof2).
demo( P1 or P2, Proof) :-
    demo( P1, Proof);
    demo( P2, Proof).

```

Código Parte B

SISTEMA DE INFERÊNCIA FORWARD CHAINING - “forward.pl”

```

%a simple backward chaining rule interpreter
:-op(800, fx, if).
:-op(700, xfx, then).
:-op(300, xfy, or).
:-op(500, xfy, and).
:-op( 800, xfx, <=).

```

```

% Simple forward chaining in Prolog
demo:-new_derived_fact(P), !, assert(fact(P)),demo.
demo:-write('Obrigado pela sua preferencia!').
new_derived_fact(Concl):-if Cond then Concl, \+ fact(Concl), composed_fact(Cond).

composed_fact(Cond):- fact(Cond).
composed_fact(Cond1 and Cond2):-
    composed_fact(Cond1),

```

```
    composed_fact(Cond2).
composed_fact(Cond1 or Cond2):-
    composed_fact(Cond1);
    composed_fact(Cond2).
```

BASE DE CONHECIMENTO- "bc.pl"

```
if max5pessoas then final:transporte_pessoal.
if mais5pessoas then final:transporte_publico.
if enjoos then final:confortavel.
if urgente then final:rapido.
if preco_baixo then final:barato.
if rapido and viagem_trabalho then final:online.
if lento then final:tomada.
```

BASE DE DADOS - "bd.pl"

```
bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,porto,30,40).
bd([barato,tomada,transporte_publico,bagagemDePorao],autocarro,braga,porto,20,30).
bd([barato,online,confortavel,transporte_pessoal,bagagemDeMao],carro,braga,porto,20,25).
bd([confortavel,transporte_publico,bagagemDeMao],aviao,braga,porto,20,70).
bd([online,confortavel,transporte_privado],taxi,braga,porto,20,50).

bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,faro,110,60).
bd([barato,tomada,transporte_publico,bagagemDePorao],autocarro,braga,faro,130,80).
bd([barato,online,confortavel,transporte_pessoal,bagagemDeMao],carro,braga,faro,100,80).
bd([confortavel,transporte_publico,bagagemDeMao],aviao,braga,faro,90,150).
bd([online,confortavel,transporte_privado],taxi,braga,faro,100,130).

bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,guimaraes,20,30).
bd([barato,transporte_publico,bagagemDePorao],autocarro,braga,guimaraes,20,25).
bd([barato,online,confortavel,transporte_pessoal,bagagemDeMao],carro,braga,guimaraes,15,20).
bd([confortavel,transporte_publico,bagagemDeMao],aviao,braga,guimaraes,10,60).
bd([online,confortavel,transporte_privado],taxi,braga,guimaraes,15,40).

bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,aveiro,70,40).
bd([barato,transporte_publico,bagagemDePorao],autocarro,braga,aveiro,70,35).
bd([barato,online,confortavel,transporte_pessoal,bagagemDeMao],carro,braga,aveiro,65,60).
.
bd([confortavel,transporte_publico,bagagemDeMao],aviao,braga,aveiro,50,90).
bd([online,confortavel,transporte_privado],taxi,braga,aveiro,65,70).

bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,lisboa,80,45).
bd([barato,tomada,transporte_publico,bagagemDePorao],autocarro,braga,lisboa,75,50).
```

```

bd([barato,online,confortavel,transporte_pessoal,bagagemDeMao],carro,braga,lisboa,70,60)
.
bd([confortavel,transporte_publico,bagagemDeMao],aviao,braga,lisboa,50,95).
bd([online,confortavel,transporte_privado],taxi,braga,lisboa,70,90).

bd([barato,confortavel,transporte_publico,bagagemDePorao],comboio,braga,coimbra,55,40).
bd([barato,tomada,transporte_publico,bagagemDePorao],autocarro,braga,coimbra,55,42).
bd([barato,online,confortavel,transporte_pessoal,bagagemDeMao],carro,braga,coimbra,50,66).
bd([confortavel,transporte_publico,bagagemDeMao],aviao,braga,coimbra,44,99).
bd([online,confortavel,transporte_privado],taxi,braga,coimbra,50,88).

```

INTERFACE - "interface.pl"

```

:-dynamic(fact/1).
:-[bd,bc,proof,forward].

%:-initialization(interface).

interface:- retractall(fact(_)), nl,nl,write('Bem-vindo ao nosso sistema baseado em
conhecimento!!'),nl,
write('Necessita de ajuda para recomendacao de meio de transporte?'),nl,
    write(' "1." - Sim, ajude-me'),nl,
    write('"0." - Nao. Quero sair '),nl,read(A),(
        (1 == A), nl,origem,nl;
        (0 == A), nl,write('Terminou'),halt).
%braga, porto,aveiro,guima,viseu,lisboa,faro,coimbra,
origem:- nl,nl,write('Qual o seu local de partida?'),nl,
    write(' "1." - Braga '),nl,
    write(' "2." - Porto '),nl,
    write(' "3." - Faro '),nl,
    write(' "4." - Guimaraes '),nl,
    write(' "5." - Aveiro '),nl,
    write(' "6." - Lisboa '),nl,
    write(' "7." - Coimbra '),nl,

    read(B),(
        (1 == B), nl,assert(origem(braga)),destino,nl;
        (2 == B), nl,assert(origem(porto)),destino,nl;
        (3 == B), nl,assert(origem(faro)),destino,nl;
        (4 == B), nl,assert(origem(guimaraes)),destino,nl;
        (5 == B), nl,assert(origem(aveiro)),destino,nl;
        (6 == B), nl,assert(origem(lisboa)),destino,nl;
        (7 == B), nl,assert(origem(coimbra)),destino,nl;

```

```
(otherwise), nl,write('Introduza uma opcao valida, por favor comece de novo!'),nl,nl,origem).
```

```
destino:- nl,nl,write('Qual o seu destino? '),nl,  
  write(' "1." - Braga '),nl,  
  write(' "2." - Porto '),nl,  
  write(' "3." - Faro '),nl,  
  write(' "4." - Guimaraes '),nl,  
  write(' "5." - Aveiro '),nl,  
  write(' "6." - Lisboa '),nl,  
  write(' "7." - Coimbra '),nl,
```

```
read(C),(  
  (1 == C), nl,assert(destino(braga)),tipo_viagem,nl;  
  (2 == C), nl,assert(destino(porto)),tipo_viagem,nl;  
  (3 == C), nl,assert(destino(faro)),tipo_viagem,nl;  
  (4 == C), nl,assert(destino(guimaraes)),tipo_viagem,nl;  
  (5 == C), nl,assert(destino(aveiro)),tipo_viagem,nl;  
  (6 == C), nl,assert(destino(lisboa)),tipo_viagem,nl;  
  (7 == C), nl,assert(destino(coimbra)),tipo_viagem,nl;  
  (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl, destino).
```

```
tipo_viagem:-nl,nl,write('Qual o seu objetivo de viagem?'),nl,  
  write(' "1." - Trabalho'),nl,  
  write(' "2." - Lazer'),nl,
```

```
read(D),(  
  (1 == D), assert(fact(viagem_trabalho)),pc,nl;  
  (2 == D), assert(fact(viagem_lazer)),estadia,nl;  
  (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,estadoViajante).
```

```
estadia:-nl,nl,write('Quanto tempo vai ficar hospedada?'),nl,  
  write('1. - Mais de uma semana'),nl,  
  write('2. - menos de uma semana '),nl,
```

```
read(G),(  
  (1 == G), assert(fact(bagagem)),estadoViajante;  
  (2 == G), assert(fact(n_bagagem)),estadoViajante;  
  (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,estadia).
```

```
pc:-nl,nl,write('Pretende usar computador durante a viagem?'),nl,  
  write('1. - Sim'),nl,
```

```

write('2. - Nao '),nl,

read(G),(
  (1 == G), assert(fact(computador)),estadoViajante;
  (2 == G), assert(fact(n_computador)),estadoViajante;
  (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl, pc).

estadoViajante:-nl,nl,write('E urgente?'),nl,
  write('1. - Quero chegar ao destino o mais cedo possivel'),nl,
  write("2. - Nao, tenho tempo"),nl,

read(D),(
  (1 == D), assert(fact(urgente)),preco,nl;
  (2 == D), assert(fact(n_urgente)),preco,nl;
  (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,
estadoViajante).

preco:-nl,nl,write('Quanto e que esta disposto a gastar?'),nl,
write('1. - O minimo possivel '),nl,
write('2. - Isso nao e problema para mim'),nl,

read(E),(
  (1 == E), assert(fact(preco_baixo)),enjoo;
  (2 == E), assert(fact(preco_alto)),enjoo;
  (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,
preco).

enjoo:-nl,nl,write('Tem enjoos?'),nl,
write('1. - Sim'),nl,
write('2. - Nao'),nl,

read(F),(
  (1 == F), assert(fact(enjoos)),fumador;
  (2 == F), assert(fact(n_enjoos)),fumador;
  (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,
enjoo).

fumador:-nl,nl,write('Fuma?'),nl,
write('1. - Sim sou fumador'),nl,
write('2. - Nao'),nl,

read(H),(
  (1 == H), assert(fact(fumador)),numero_pessoas;
  (2 == H), assert(fact(n_fumador)),numero_pessoas;
  (otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,
fumador).

```

```

numero_pessoas:-
    nl, write('Quantas pessoas vao consigo na viagem? '), nl,
    read(N),
    (N > 5, assert(fact(mais5pessoas)), resultado;
    N <= 5, assert(fact(max5pessoas)), resultado;
    (otherwise), nl, write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,
    numero_pessoas).

```

resultado :- nl,nl,demo, comparar.

```

comparar :-
    origem(Origem),
    destino(Destino),
    findall(bd(L, M, Origem, Destino, Distancia, Custo), bd(L, M, Origem,
Destino, Distancia, Custo), K),
    resposta(K), fim.

```

```

resposta([]) :-
    write('Esperemos que goste da recomendacao, aproveite!'), nl, write('Se nao houver
recomendacoes disponiveis no momento, tente novamente mais tarde.').

```

```

resposta([bd(X, T, O, D, M, C) | R]) :-
    findall(B, fact(final:B), L),
    subset(L, X), nl, nl,
    write('_____'), write('Meio de transporte mais recomendado '), write('_____
'), nl, nl, write('_____'), write(T), nl, nl, nl,
    write('-> '), write('Origem: '), write(O), nl, nl,
    write('-> '), write('Destino: '), write(D), nl, nl,
    write('-> '), write('Distancia: '), write(M), nl, nl,
    write('-> '), write('Custo: '), write(C), nl, nl, nl,

    resposta(R).

```

```

resposta([_|R]) :-
    resposta(R).

```

```

fim :-
    retractall(fact(_)),
    retractall(origem(_)),
    retractall(destino(_)), nl, nl, nl,
    write('Obrigado pela sua preferencia! Deseja ser recomendado novamente?'), nl, nl,
    write(' "1." - Sim, por favor'), nl,
    write(' "0." - Nao. Quero sair '), nl,
    read(Z), (
        (1 == Z), nl, retractall(fact(_)), origem;
        (0 == Z), nl, write('Terminou'), halt;
    ).

```

```
(otherwise), nl,write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl,
fim).
```

PROOF - “proof.pl”

```
% demo( P, Proof), where Proof is a proof that P is true
:- op( 800, fx, if).
:- op( 700, xfx, then).
:- op( 300, xfy, or).
:- op( 500, xfy, and).
:- op( 800, xfx, <=).
```

```
demo( P, P) :- fact( P).
demo( P, P <= CondProof) :-
    if Cond then P, demo( Cond, CondProof).
demo( P1 and P2, Proof1 and Proof2) :-
    demo( P1, Proof1), demo( P2, Proof2).
demo( P1 or P2, Proof) :-
    demo( P1, Proof);
    demo( P2, Proof).
```


Anexo B

Contrato do Grupo

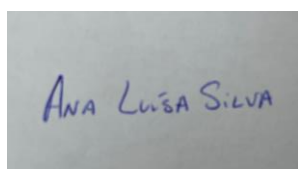
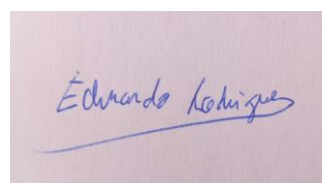
Por forma de assegurar o funcionamento correto do grupo, foram acordadas várias regras, que todos os elementos do grupo se responsabilizam por cumprir na íntegra:

- 1 - No início de cada semana faz-se uma reunião de presença obrigatória, exceto em casos devidamente justificados e aceites por todos os membros do grupo.
- 2 - Cada elemento do grupo deve realizar a tarefa que lhe foi atribuída, em caso de tarefas individuais, e deve comparecer a todas as reuniões extraordinárias para realização de tarefas conjuntas, exceto em casos devidamente justificados e aceites por todos os membros do grupo.
- 3 - A divisão de tarefas deve ser feita por forma a distribuir o mais igualmente possível o nível de dificuldade e esforço que cada tarefa necessita.
- 4 - Quando existem dúvidas na realização de uma tarefa individual, o elemento em questão é obrigado a informar os restantes membros dessa dúvida para a resolver ou se necessária marcação de reunião extraordinária.
- 5 - Cada elemento irá cumprir os prazos delineados para as suas tarefas, quando individuais, fazendo para isso uso da cláusula 4 quando necessário.
- 6 - O não cumprimento das cláusulas acima descritas, o desrespeito, falta de interesse, incumprimento de prazos ou outras falhas de carácter de igual natureza por qualquer elemento do grupo implicam a total desvalorização do seu contributo para o grupo, e consequentemente atribuição de nota 0 (zero) na sua nota final do projeto.
- 7 - Este contrato é válido e considera-se vinculativo aquando da assinatura de todos os elementos do grupo.

Ana Luísa Silva

André Macedo Barros

Carlos Rodrigues

A photograph of a white piece of paper with the name "ANA LUÍSA SILVA" written in blue ink.A photograph of a white piece of paper with a stylized handwritten signature in blue ink.A photograph of a white piece of paper with the name "Carlos Rodrigues" written in blue ink.

Bibliografia

- Manual SWI-Prolog : https://www.swi-prolog.org/pldoc/doc_for?object=manual;
- Introdução à Programação Prolog, Luiz Palazzo;
- Material fornecido pelo docente nas aulas teóricas da Unidade Curricular;
- Bratko, Ivan, Prolog – PROLOG PROGRAMMING FOR ARTIFICIAL INTELLIGENCE, Longman, 2000